

Codificação Para Front-end

20

2026

SENAI - CRTI



Projeto

Carrinho de Compras com JavaScript

Consolidando todos os conhecimentos do curso em um e-commerce front-end funcional, com carrinho de compras, persistência local e interação completa.

DESENVOLVIMENTO FRONT-END

Situação de Aprendizagem

Onde estamos no projeto?

O que já construímos

Ao longo das aulas anteriores, desenvolvemos as bases do nosso e-commerce: estrutura HTML semântica, estilização com CSS responsivo, renderização dinâmica de produtos com JavaScript e manipulação do DOM com eventos de interatividade.

O problema de hoje

O e-commerce exibe produtos e permite interação, mas **ainda não deixa o usuário selecionar produtos para compra**. Falta a peça central de qualquer loja online: o carrinho de compras.

-  Sem o carrinho, o usuário navega mas não consegue comprar nada!

Capacidades que vamos desenvolver



Lógica de Carrinho

Implementar toda a lógica de adicionar, remover e atualizar itens usando arrays de objetos em JavaScript puro.



Eventos e DOM

Trabalhar com eventos de clique, delegação de eventos e manipulação dinâmica do DOM para atualizar a interface em tempo real.



localStorage

Utilizar o localStorage do navegador para persistir o carrinho, mantendo os dados mesmo após recarregar a página.



Integração Total

Unir todos os módulos do projeto em um sistema coeso e funcional, simulando o comportamento real de uma loja online.



O que falta em um e-commerce real?

1

Visualizar Produtos

Já implementado ✓

2

Selecionar Itens

Adicionar ao carrinho 🎯

3

Revisar Pedido

Ver itens e quantidades 🎯

4

Finalizar Compra

Calcular total 🎯

Conceito Fundamental

O que é um Carrinho de Compras?

No mundo real, um carrinho de supermercado é um espaço temporário onde você coloca produtos antes de decidir comprar.

No front-end, o carrinho é simplesmente um **array de objetos JavaScript** que armazena temporariamente os produtos selecionados.

Cada produto no carrinho é um objeto com: id, nome, preço, imagem e quantidade.

Analogia prática



Carrinho físico

Cesta com produtos



Carrinho digital

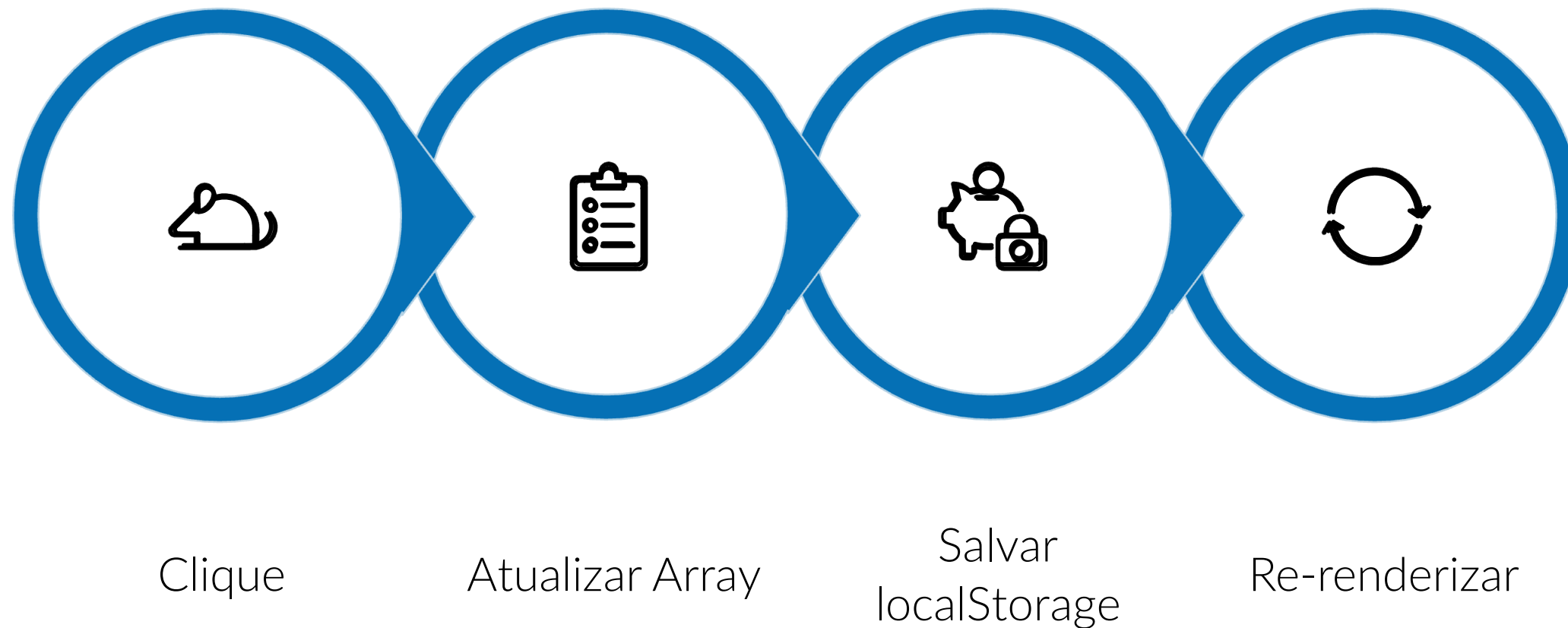
Array de objetos



O carrinho vive na memória do navegador — sem banco de dados!

Sincronização: Dados ↔ Interface

Um princípio fundamental do desenvolvimento front-end é: **a interface deve sempre refletir o estado dos dados**. Nunca modifique o HTML diretamente sem antes atualizar o array.



Seguindo este fluxo consistente, garantimos que os dados e a interface nunca ficam fora de sincronia. É a base do pensamento reativo que frameworks modernos como React e Vue adotam.

Renderizando produtos + event click

Criando produtos interativos

Mostramos produtos dinamicamente e ligamos um evento no botão..

❗ Cada botão recebe um evento próprio.

```
function renderizarProdutos() {
  productList.innerHTML = "";

  produtos.forEach(produto => {
    const card =
      document.createElement("div");

    card.classList.add("product-card");

    card.innerHTML = `
      
      <h3>${produto.nome}</h3>
      <p>R$ ${produto.preco}</p>
      <button>Adicionar ao carrinho</button>
    `;

    card
      .querySelector("button")
      .addEventListener("click", () => {
        adicionarAoCarrinho(produto.id);
      });
    productList.appendChild(card);
  });
}
```

Estado do carrinho

Guardando produtos no array

Carrinho = memória temporária da compra.

- 📘 Buscamos o produto e colocamos dentro do array carrinho.

```
let carrinho = [];  
  
function adicionarAoCarrinho(id) {  
  const produto =  
    produtos.find(p => p.id === id);  
  
  carrinho.push(produto);  
  
  salvarCarrinho();  
}
```

Renderização do carrinho + total

Atualizando interface automaticamente

❗ Mudou o array → renderiza novamente.

```
function renderizarCarrinho() {
  cartList.innerHTML = "";
  let total = 0;

  carrinho.forEach((item, index) => {
    total += item.preco;
    const li =
      document.createElement("li");
    li.innerHTML = `
      ${item.nome} - R$ ${item.preco}
      <button>Remover</button>
    `;
    li.querySelector("button")
      .addEventListener("click", () => {
        carrinho.splice(index, 1);
        salvarCarrinho();
      });
    cartList.appendChild(li);
  });

  totalEl.textContent =
    `Total: R$ ${total}`
;}
```

localStorage + inicialização

Salva no navegador

 Ao abrir a página, tudo volta como estava.

```
function salvarCarrinho() {  
  localStorage.setItem(  
    "carrinho",  
    JSON.stringify(carrinho)  
  );  
  
  renderizarCarrinho();  
  atualizarMiniCarrinho();  
}  
  
document.addEventListener(  
  "DOMContentLoaded", () => {  
    carrinho =  
      JSON.parse(  
        localStorage.getItem("carrinho")  
      ) || [];  
  
    renderizarProdutos();  
    renderizarCarrinho();  
    atualizarMiniCarrinho();  
  }  
);
```

Finalizar compra

Validação final

 Validamos antes de concluir

```
function finalizarCompra() {  
    if (carrinho.length === 0) {  
        mensagemCompra.textContent =  
            "Seu carrinho está vazio!";  
        mensagemCompra.style.color = "red";  
        return;  
    }  
  
    mensagemCompra.textContent =  
        "Compra realizada com sucesso!";  
    mensagemCompra.style.color = "green";  
  
    carrinho = [];  
    salvarCarrinho();  
}
```

Desafios Extras para ir Além

Concluiu o projeto base? Experimente esses desafios para enriquecer ainda mais o seu e-commerce:

Controle de Quantidade

Em vez de remover direto, adicione botões **+** e **-** para incrementar ou decrementar a quantidade de cada item. Remova automaticamente quando chegar a zero.

Contador no Ícone

Exiba um badge vermelho no ícone do carrinho mostrando a quantidade total de itens. Atualize dinamicamente a cada adição ou remoção.

Feedback Visual

Adicione uma animação ou mensagem de confirmação quando um produto é adicionado ao carrinho, melhorando a experiência do usuário.

Finalizar Pedido

Crie um botão "Finalizar Compra" que exhibe um resumo do pedido em um modal e limpa o carrinho após a confirmação.



Resultado Esperado

Parabéns! E-commerce Completo

Estrutura

HTML semântico e organizado com seções bem definidas

Estilo

CSS responsivo com layout adaptável a diferentes telas

Interação

JavaScript puro com DOM, eventos e manipulação de arrays

Persistência

localStorage garantindo que os dados sobrevivem ao reload

Ao concluir esta aula, você terá desenvolvido um **e-commerce front-end funcional completo**, consolidando todos os conhecimentos do curso em um projeto real e apresentável.

Métodos JavaScript Utilizados no Projeto

Método	Onde usamos	O que faz
<code>find()</code>	adicionarAoCarrinho	Busca o primeiro item que satisfaz uma condição
<code>push()</code>	adicionarAoCarrinho	Adiciona um novo item ao final do array
<code>filter()</code>	removerDoCarrinho	Retorna novo array excluindo itens indesejados
<code>forEach()</code>	renderizarCarrinho	Itera sobre cada item para gerar o HTML
<code>reduce()</code>	atualizarTotal	Acumula valores para calcular o total
<code>JSON.stringify()</code>	salvarNoLocalStorage	Converte array em string para salvar
<code>JSON.parse()</code>	carregarDoLocalStorage	Converte string de volta para array

Reflexão Final

O que você construiu é real 💡

Você não apenas aprendeu sintaxe — você construiu um produto funcional. Um e-commerce com carrinho de compras é exatamente o tipo de projeto que aparece em entrevistas e portfólios profissionais.



Portfólio

Publique no GitHub Pages e compartilhe o link



LinkedIn

Adicione o projeto ao seu perfil profissional



Evolução

Continue incrementando com novos recursos

